10 优化分支预测

到目前为止，我们一直在讨论优化内存访问和计算。然而，还有另一个重要的性能瓶颈类别，我们尚未讨论。它与推测执行有关，这是现代高性能 CPU 核心中普遍存在的一项功能。为了提醒你，可以参考 Section 3.3.3 中我们讨论了如何利用推测执行来提高性能。在本章中，我们将探讨减少分支预测错误次数的技术。

一般来说，现代处理器非常擅长预测分支结果。它们不仅遵循静态预测规则，还检测动态模式。通常，分支预测器保存先前分支结果的历史记录，并尝试猜测下一个结果。然而，当模式变得难以跟踪时，CPU 分支预测器可能会影响性能。

当分支预测错误时，会导致显著的速度惩罚。当这种事件经常发生时，CPU 需要清除所有预测性工作，后来证明是错误的。它还需要清空流水线，并开始填充正确路径的指令。通常，现代 CPU 由于分支预测错误而经历 10 到 20 个周期的惩罚。准确的周期数取决于微架构设计，即流水线的深度和从错误预测中恢复的机制。

分支预测器使用缓存和历史寄存器，因此容易受到与缓存相关的问题的影响，即三个 C：

- **强制性缺失（Compulsory misses）**：当采用静态预测时，如果在分支的第一次动态出现时没有可用的动态历史信息，可能会发生误预测。
- **容量缺失（Capacity misses）**：由于程序中分支数量极高或动态模式非常长，导致动态历史丢失，从而产生误预测。
- **冲突缺失（Conflict misses）**：分支通过它们的虚拟地址和/或物理地址的组合被映射到缓存桶（关联集合）中。如果太多活跃的分支被映射到同一个集合，就可能发生历史丢失。冲突缺失的另一个例子是虚伪共享，当两个独立的分支被映射到同一个缓存条目时，它们可能会相互干扰，从而可能降低预测历史的质量。

程序总是会有非零数量的分支误预测。你可以通过查看 TMA（Top-Down Microarchitecture Analysis）的`Bad Speculation`（错误推测）指标来了解程序受分支误预测的影响程度。对于通用应用程序来说，`Bad Speculation`指标在 5-10% 的范围内是正常的。我们建议一旦这个指标超过 10%，就要密切关注。

由于分支预测器擅长发现模式，过去优化分支预测的老建议已经不再适用。过去，开发者可以选择以分支指令的编码前缀形式向处理器提供预测提示（例如`0x2E: Branch Not Taken`, `0x3E: Branch Taken`）。这可能在旧的微架构上，如 Pentium 4 上提高性能。虽然使用这些分支前缀仍然可以得到有效的 x86/x64 汇编代码，但在现代处理器上不会产生性能提升。

减少分支误预测的一个间接方法是使用基于源代码和编译器的技术来简化代码。PGO（Profile-Guided Optimization，基于性能分析的优化）和 BOLT（一种优化工具）通过提高 fallthrough 率来减少分支误预测，从而减轻分支预测器结构的压力。我们将在下一章讨论这些技术。

所以，也许直接消除分支误预测的唯一方法就是消除分支本身。在接下来的两个部分中，我们将探讨如何用查找表和预测来替换分支。

有一个传统观点认为，永远不会执行的分支对分支预测是透明的，不会影响性能，因此从预测的角度来看，移除它们并没有太大意义。然而，与这一观点相反，BOLT 优化器的作者进行的实验表明，在大型代码足迹的应用程序中，如 Clang C++ 编译器，用等大小的 nop 指令替换永远不会执行的分支，可以在现代 Intel CPU 上带来大约 5% 的速度提升。所以，尝试消除所有分支仍然是值得的。

10.1 用查找表替换分支 (Replace Branches with Lookup)

避免频繁出现分支预测错误的一种方法是使用查找表。在 Listing 10.1中展示了一个可能会受益于这种转换的代码示例。与往常一样，原始版本在左侧，改进版本在右侧。函数 `mapToBucket` 将 `[0-50)` 范围内的值映射到对应的五个

桶中，并对超出此范围的值返回 -1。对于均匀分布的 v 值，v 有相等的概率落入任何一个桶中。在原始版本的生成的汇编中，我们可能会看到许多分支，这可能会导致高的分支预测错误率。希望可以通过单个数组查找来重写函数 mapToBucket，如右侧所示。

代码清单：用查找表替换分支。

```
int8_t mapToBucket(unsigned v) {
  if (v >= 0  && v < 10) return 0;
  if (v >= 10 && v < 20) return 1;
  if (v >= 20 && v < 30) return 2;
  if (v >= 30 && v < 40) return 3;
  if (v >= 40 && v < 50) return 4;
  return -1;
}
```

```
=>
```

```
int8_t buckets[50] = {
  0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
  1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
  2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
  3, 3, 3, 3, 3, 3, 3, 3, 3, 3,
  4, 4, 4, 4, 4, 4, 4, 4, 4, 4};

int8_t mapToBucket(unsigned v) {
  if (v < (sizeof(buckets) / sizeof(int8_t)))
    return buckets[v];
  return -1;
}
```

对于右侧改进版本的 mapToBucket，编译器很可能会生成一个单个分支指令，以防止对 buckets 数组进行越界访问。这个函数的典型热路径将执行未采取的分支和一个加载指令。由于我们期望大多数输入值都落在 buckets 数组覆盖的范围内，CPU 分支预测器将很好地预测这个分支。查找速度也很快，因为 buckets 数组很小且很可能位于 L1-d 缓存中。

如果我们需要映射一个更大范围的值，比如 [0-1M)，分配一个非常大的数组是不现实的。在这种情况下，我们可以使用区间映射数据结构，它们可以使用更少的内存来实现这个目标，但查找复杂度是对数级的。读者可以在 Boost[179] 和 LLVM[180] 找到现有的区间映射容器实现。

10.2 用算术替换分支 (Replace Branches with Arithmetic)

在某些情况下，可以用算术来替换分支。如 Listing 10.1中的代码，也可以使用简单的算术公式重写，如 Listing 10.2所示。对于这段代码，Clang-17 编译器用更便宜的乘法运算替换了昂贵的除法运算。

代码清单: 用算术替换分支。

```
int8_t mapToBucket(unsigned v) {
  constexpr unsigned BucketRangeMax = 50;
  if (v < BucketRangeMax)
    return v / 10;
  return -1;
}
```

截至 2023 年，编译器通常无法自行找到这些捷径，因此由程序员手动完成。如果你能找到用算术替换分支的方法，你很可能会看到性能改进。不幸的是，这并不总是可能的。

[179] C++ Boost `interval_map` - https://www.boost.org/doc/libs/1_65_0/libs/icl/doc/html/boost/icl/interval_map.html

[180] LLVM's `IntervalMap` - https://llvm.org/doxygen/IntervalMap_8h_source.html

10.3 用谓词替换分支 (Replace Branches with Predication)

某些分支可以通过执行分支的两部分，然后选择正确的结果（谓词）来有效地消除。当这种转换可能有利可图时，代码示例显示在 Listing 10.3 中。如果 TMA 提示 `if (cond)` 分支具有非常高的误判率，您可以尝试通过执行右侧显示的转换来消除分支。

代码清单：谓词分支。

```
int a;                                          int x = computeX();
if (cond) { /* frequently mispredicted */   =>  int y = computeY();
  a = computeX();                               int a = cond ? x : y;
} else {
  a = computeY();
}
```

对于右侧的代码，编译器可以替换来自三元运算符的分支，并生成 CMOV x86 指令。CMOVcc 指令检查 EFLAGS 寄存器（CF、OF、PF、SF 和 ZF）中一个或多个状态标志的状态，并在标志处于特定状态或条件下执行移动操作。可以使用 FCMOVcc,VMAXSS/VMINSS 指令对浮点数字进行类似的转换。Listing 10.3 显示了原始版本和无分支版本的汇编列表。

代码清单：谓词分支 - x86 汇编代码。

```
# original version                  # branchless version
400504: test edi,edi                400537: mov eax,0x0
400506: je 400514                   40053c: call <computeX> # compute x; a = x
400508: mov eax,0x0                 400541: mov ebp,eax     # ebp = x
40050d: call <computeX>       =>    400543: mov eax,0x0
400512: jmp 40051e                  400548: call <computeY> # compute y; a = y
400514: mov eax,0x0                 40054d: test ebx,ebx    # test cond
400519: call <computeY>             40054f: cmovne eax,ebp  # override a with x if needed
40051e: mov edi,eax
```

与原始版本相比，无分支版本没有跳转指令。然而，无分支版本独立计算 x 和 y，然后选择其中一个值并丢弃另一个值。虽然这种转换消除了分支预测错误的惩罚，但它可能比原始代码做了更多的工作。在这种情况下，性能改进在很大程度上取决于 computeX 和 computeY 函数的特性。如果函数很小，编译器能够内联它们，那么它可能会带来明显的性能提升。如果函数很大，执行这两个函数的成本可能比承担分支预测错误的成本更低。

需要注意的是，谓词并不总是能提高应用程序的性能。谓词的问题在于它限制了 CPU 的并行执行能力。对于原始版本的代码，CPU 可以预测分支将被取走，推测性地调用 computeX 并继续执行程序的其余部分。对于无分支版本，这种类型的推测是不可能的，因为 CPU 必须等待 CMOVNE 指令的结果才能继续进行。

在选择代码的常规版本和无分支版本之间进行权衡时，典型的例子是二分查找[181]：

- **对于无法放入 CPU 缓存的大型数组的搜索，基于分支的二分查找版本性能更好**，因为分支预测错误的惩罚与内存访问的延迟相比很低（由于缓存未命中，延迟很高）。由于存在分支，CPU 可以推测其结果，从而允许同时从当前迭代和下一个迭代加载数组元素。它并没有就此结束：推测仍在继续，您可能同时有多个加载正在进行。
- **对于适合 CPU 缓存的小型数组，情况则相反**。正如前面所解释的，无分支搜索仍然将所有内存访问序列化。但这一次，加载延迟很小（只有少数周期），因为数组适合 CPU 缓存。基于分支的二分查找会不断遭受误判，其成本大约为 10-20 个周期。在这种情况下，误判的成本远高于内存访问的成本，因此推测执行的优势会受到阻碍。无分支版本通常在这种情况下更快。

[181] 关于无分支二分查找的讨论 - https://stackoverflow.com/a/54273248。

二分查找是一个很好的例子，它展示了如何在选择标准实现和无分支实现时进行推理。现实世界的场景可能更难分析，因此，再次测量以找出在您的情况下替换分支是否有益。

如果没有性能分析数据，编译器就无法了解误判率。因此，编译器通常默认生成分支（即原始版本）。它们在使用谓词方面比较保守，即使在简单的情况下也可能抵制生成 CMOV 指令。同样，权衡也很复杂，如果没有运行时数据，就很难做出正确的决定。基于硬件的 PGO（参见 [#sec:secPGO]）将是这里的一大进步。此外，还有一种方法可以通过硬件机制向编译器指示分支条件是不可预测的。从 Clang-17 开始，编译器现在支持 `__builtin_unpredictable`，它可以非常有效地将不可预测的分支替换为 CMOV x86 指令。例如：

```
if (__builtin_unpredictable(x != 2))
  y = 0;
if (__builtin_unpredictable(x == 3))
  y = 1;
```

问题和练习

1. 使用本章讨论的技术解决以下实验作业：
   - `perf-ninja::branches_to_cmov_1`
   - `perf-ninja::lookup_tables_1`
   - `perf-ninja::virtual_call_mispredict`
   - `perf-ninja::conditional_store_1`
2. 运行您每天使用的应用程序。收集 TMA 细分并检查 `BadSpeculation` 指标。查看代码中哪个部分归因于最多的分支预测错误。是否可以使用本章讨论的技术来避免分支？

AI 回答：

1. 对于提到的实验作业，以下是如何使用本章讨论的技术来解决的概述：

   - `perf-ninja::branches_to_cmov_1`：这个实验可能涉及将条件分支转换为条件移动（CMOV）指令。在这种情况下，你可以使用谓词替换分支的技术，通过执行两个可能的路径（例如，`computeX` 和 `computeY`），然后使用条件移动指令（如 `cmovne` 或 `cmovle`）来选择正确的结果。这需要编译器支持或者程序员手动重写代码。

   - `perf-ninja::lookup_tables_1`：这个实验可能要求使用查找表来替换分支。你可以创建一个数组或哈希表，其中包含了预先计算的结果，然后通过索引直接获取结果，从而避免使用分支。这适用于那些有固定范围和可预测模式的输入。

   - `perf-ninja::virtual_call_mispredict`：这个实验可能关注于虚拟函数调用（如在 C++ 中的虚函数）导致的分支预测错误。优化方法可能包括使用内联函数、减少虚函数的使用，或者使用更高效的分发机制，如基于范围的分发。

   - `perf-ninja::conditional_store_1`：这个实验可能涉及条件存储操作，即根据条件执行数据的存储。可以通过使用条件执行的汇编指令（如 `cmov` 系列指令）来优化，或者在可能的情况下，重构代码以避免条件存储。

2. 要收集 TMA（Top-Down Microarchitecture Analysis）细分并检查`BadSpeculation`指标，你需要使用性能分析工具，如 Linux 的`perf`工具。这通常涉及到以下步骤：

   - 使用性能分析工具运行你的应用程序。
   - 收集和分析性能数据，特别是关注分支预测错误的指标。
   - 识别导致最多分支预测错误的代码部分。

一旦你确定了问题区域，你可以应用本章讨论的技术来尝试优化。例如，如果一个特定的循环或条件语句导致了高分支预测错误率，你可以考虑使用查找表、算术替换或谓词替换来重写这部分代码。然而，是否应用这些技术需要根据实际情况和性能分析的结果来决定，因为并非所有情况下这些优化都能带来性能提升。在某些情况下，优化可能会增加代码复杂性或降低可读性，因此需要权衡利弊。

编码练习：编写一个微基准，使其经历 50% 的预测错误率或尽可能接近 50%。您的目标是用代码编写，使其所有分支指令中有一半被预测错误。这并不像您想象的那么简单。一些提示和想法：

- 分支预测错误率计算为 `BR_MISP_RETIRED.ALL_BRANCHES / BR_INST_RETIRED.ALL_BRANCHES`。
- 如果使用 C++ 编码，您可以 1) 使用类似于 perf-ninja 的 google benchmark；或者 2) 编写常规控制台程序并使用 Linux `perf` 收集 CPU 计数器；或者 3) 将 libpfm 集成到微基准中（参见 Section 5.4）。
- 无需发明复杂的算法。一种简单的方法是在范围 `[0;100)` 中生成一个伪随机数，并检查它是否小于 50。随机数可以提前预生成。
- 请记住，现代 CPU 可以记住较长（但仍然有限）的分支结果序列。

chatGPT 回答：

编写一个微基准来实现 50% 的分支预测错误率并不是一件容易的事情，因为现代处理器会根据历史数据进行分支预测，使得准确达到这个目标比较困难。不过，我们可以尝试一种简单的方法来接近这个目标：通过随机生成分支指令，来达到近似的错误率。

下面是一个简单的 C++ 示例，使用 Google Benchmark 库来编写微基准：

```
#include <benchmark/benchmark.h>
#include <cstdlib>
#include <ctime>

// 定义一个简单的函数，其中包含分支预测错误率接近 50%
bool branchPredictionBenchmark() {
    // 随机生成一个数
    int randomNum = rand() % 100;
    // 如果随机数小于 50，则返回 true，否则返回 false
    return randomNum < 50;
}

// 定义微基准
static void BM_BranchPrediction(benchmark::State& state) {
    // 循环执行微基准
    for (auto _ : state) {
        // 调用函数并记录结果
        bool result = branchPredictionBenchmark();
        // Do something with result to prevent optimization
        benchmark::DoNotOptimize(result);
    }
}
// 注册微基准
BENCHMARK(BM_BranchPrediction);

// 运行微基准
```

```
BENCHMARK_MAIN();
```

这个微基准中，`branchPredictionBenchmark` 函数会随机生成一个数，并返回一个布尔值，如果生成的数小于 50，则返回 true，否则返回 false。这样，就近似实现了 50% 的分支预测错误率。

请确保在编译时包含 Google Benchmark 库，并使用合适的编译选项。

章节总结

- 现代处理器在预测分支结果方面非常出色。因此，我们建议只有当 TMA 报告显示高 `Bad Speculation` 指标时才开始修复分支预测错误的工作。
- 当分支结果模式变得难以让 CPU 分支预测器跟踪时，应用程序的性能可能会受到影响。在这种情况下，无分支版本的算法可能表现更好。本章展示了如何用查找表、算术和谓词替换分支。在某些情况下，还可以使用编译器内联函数消除分支，如 [Kapoor, 2009] 所示。
- 无分支算法并不是普遍有利的。始终测量以找出最适合您特定情况的方法。